RC1: WSL Tutorial

Windows Command line tools (WSL)

The Windows Subsystem for Linux (WSL) runs an Ubuntu Linux guest virtual machine on your Windows host machine.

Notice

- If you think something will go wrong, please **back up your data** before any operations.
- Everyone's computer differs. If you meet errors when installing WSL, go to the **Microsoft Troubleshooting section**, or search the error message on the Internet. This will help you improve your **self-learning ability**.

Upgrade Windows

We recommend **Windows 11**. Windows 10 **version 2004 build 19041 and higher** will also work.

> **Pitfall**
> Make sure Windows Update is enabled and your machine is up-to-date. Search for **"Windows Update"** in the start menu.

⚠ **Pitfall:** Make sure Windows Update is enabled and your machine is up-to-date. Search for "Windows Update" in the start menu.

Install WSL

Start **PowerShell** and run it as administrator. Search for PowerShell in the start menu, then right-click and select **"Run as administrator"**.

PowerShell is a command-line interface for Windows. Its command prompt ends with a `>`.

To follow the steps below, type the commands that appear after `>` and hit Enter.

Check if WSL is already installed:

```
wsl -l -v
```

If you see output like this, WSL is already installed:

```
C:\WINDOWS\system32> wsl -l -v
  NAME                   STATE           VERSION
* Ubuntu-24.04           Running         2
```

If the **VERSION** is `1`, you must upgrade to **WSL 2**. If you already have an Ubuntu distribution with **VERSION 2**, skip to **Setup Ubuntu** below. Otherwise, WSL is not installed and you'll see help text similar to the following.

```
C:\WINDOWS\system32> wsl -l -v
Copyright (c) Microsoft Corporation. All rights reserved.
Usage: wsl.exe [Argument]
...
```

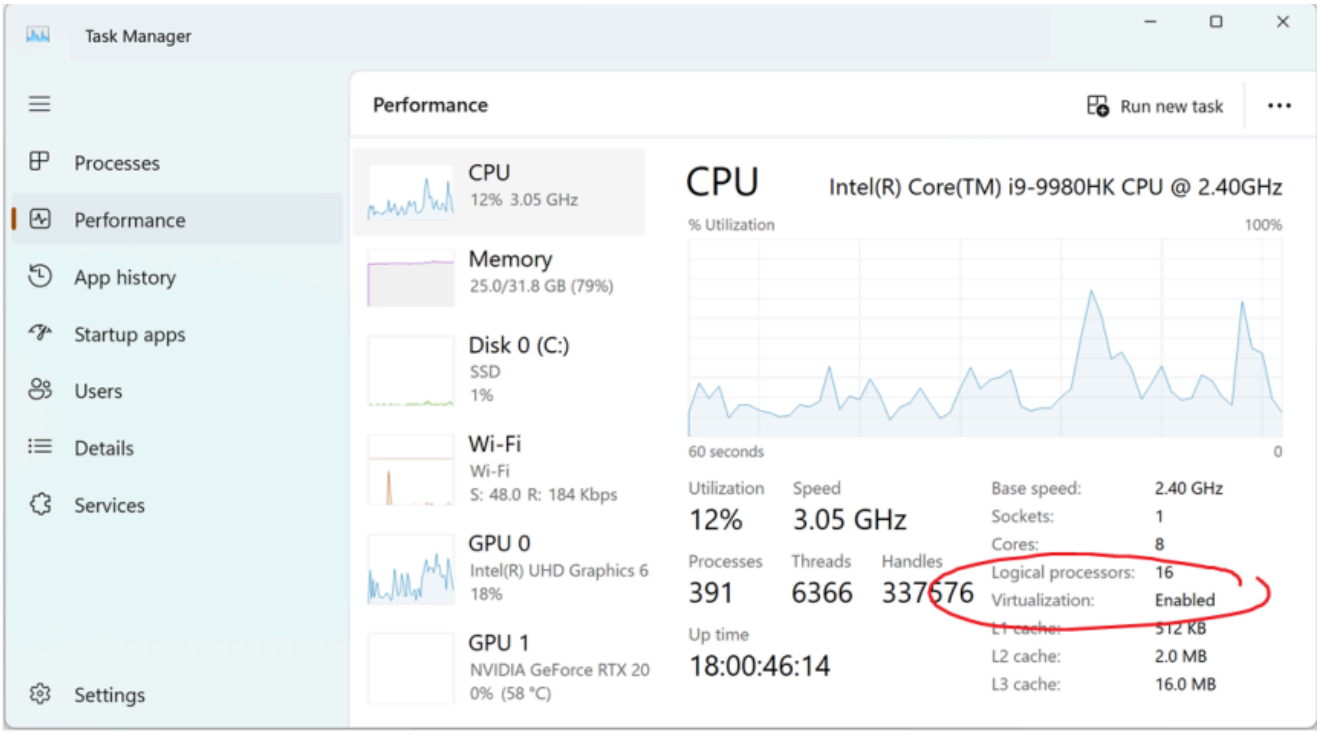
Install WSL. We recommend the most recent Ubuntu LTS version from the Microsoft App Store, currently **Ubuntu 24.04**.

```
wsl --install -d Ubuntu-24.04
```

```
C:\WINDOWS\system32> wsl --install -d Ubuntu-24.04
```

Launch **Task Manager** by pressing `Ctrl + Shift + Esc`. Click on the **Performance** tab.

If you see **"Virtualization: Enabled"**, proceed to step 2. If you do not see **"Virtualization"** at all, your computer may not support WSL. If you see **"Virtualization: Disabled"**, you need to enable virtualization in BIOS settings. The method varies by manufacturer, but instructions are generally available online. After restarting into BIOS, navigate the options with your keyboard to find and enable virtualization. Save your changes and restart your computer.



Once virtualization is enabled, ensure the **Virtual Machine Platform** feature is enabled in Windows. Search for **"Turn Windows features on or off"** in the start menu. Scroll down and ensure that **"Virtual Machine Platform"** is checked. Also ensure that **"Windows Subsystem for Linux"** is checked. You may be asked to restart your computer.

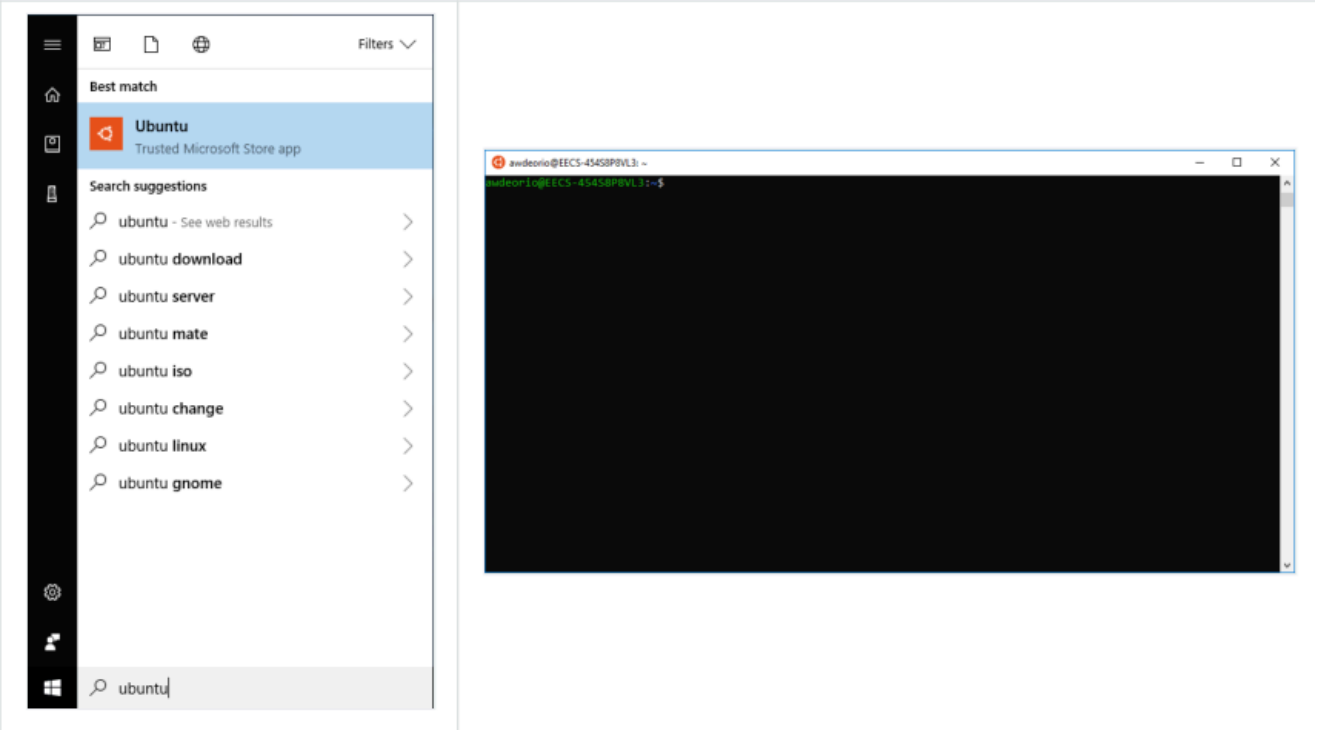
After all is done, restart your computer.

Start PowerShell as administrator, and check that WSL Ubuntu is now installed with version 2:

```
C:\WINDOWS\system32> wsl -l -v
  NAME                   STATE           VERSION
* Ubuntu-24.04           Running         2
```

Setup Ubuntu

From the previous section, you have installed WSL and an Ubuntu Linux distribution. You'll do most C++ development in Ubuntu rather than Windows. Let's get it set up. Start an Ubuntu Bash shell. Search for **"Ubuntu"** (or **"Ubuntu 24.04 LTS"**) or **"bash"** in the start menu.



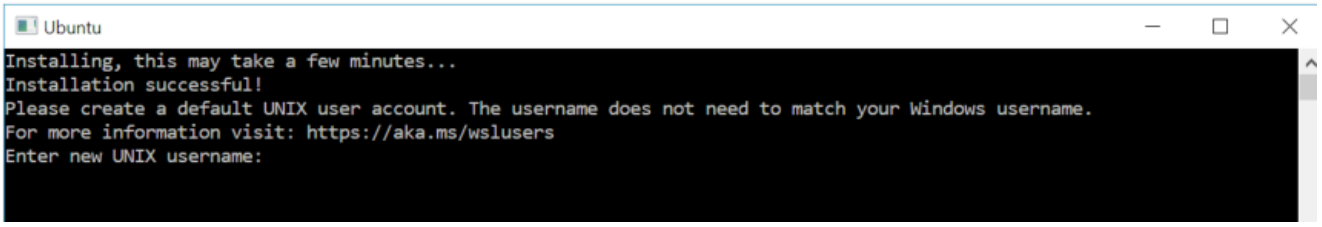
The bash command prompt ends with a `$`. To follow the steps below, type commands that appear after the `$` and hit Enter.

Create Account

The first time you open Ubuntu, you will be prompted to create a Linux username and password. Follow the prompts to create one.

- Choose a username that is all lowercase characters.

- While typing your password, the characters are not shown. This is fine.
- You can press `Enter` to skip fields other than username and password.



Double check that your user has been created:

```
whoami
```

This should **not** say `root`.

> **Pitfall**
> If you are signed in as `root`, something has gone wrong. This can happen, for example, if you accidentally closed the terminal window before creating a user account. We recommend you completely reinstall Ubuntu. (Note this will delete all of the data associated with Ubuntu. If you've just installed it, that's fine.)

Open PowerShell and run it as administrator. Then run:

```
wsl --unregister Ubuntu-24.04
wsl --install -d Ubuntu-24.04
```

```
$ whoami
awdeorio  # this should NOT say root
```

⚠ **Pitfall:** If you are signed in as root, something has gone wrong.

```
$ whoami
root  # SOMEthiNG IS WRONG
```

This can happen, for example, if you accidentally closed the terminal window before creating a user account.

We recommend you completely reinstall Ubuntu. (Note this will *delete all of the data associated with Ubuntu*. If you've just installed it, that's fine.)

Open PowerShell and run it as administrator. Run the commands below. You'll be prompted to create a user account.

```
C:\WINDOWS\system32> wsl --unregister Ubuntu-24.04
Unregistering.
The operation completed successfully.

C:\WINDOWS\system32> wsl --install -d Ubuntu-24.04
Ubuntu 24.04 LTS is already installed.
Launching Ubuntu 24.04 LTS...
Installing, this may take a few minutes...
Please create a default UNIX user account. The username does not need to match your Windows username.
For more information visit: https://aka.ms/wslusers
Enter a new UNIX username:
```

Install CLI tools

From an Ubuntu bash terminal, use the `apt` package manager to install a few command line programs:

```
sudo apt update
sudo apt install g++ make rsync wget git ssh gdb python3 tree
```

```
$ sudo apt update
$ sudo apt install g++ make rsync wget git ssh gdb python3 tree
```

Home Directory

You'll generally want to store your coding projects in the Linux filesystem, which is separate from your Windows filesystem. Open an Ubuntu bash terminal and run:

```
cd ~
pwd
```

This will take you to your Ubuntu home directory.

```
$ cd ~
$ pwd
/home/jjuett
```

Create a folder by running:

```
mkdir VE280
ls
```

I highly recommend you store all coding work for VE280 here.

Avoid paths that contain spaces. Spaces cause problems with some command line tools.

> **Pitfall**
> Linux (Ubuntu) has a separate home directory. Storing code in your Windows home directory can cause slowdowns.

⚠️ **Pitfall:** Linux (Ubuntu) has a separate home directory. Storing code in your Windows home directory can cause slowdowns.

```
$ pwd
/home/awdeorio ...          # Good, Linux home
/c/mnt/Users/awdeorio ...   # Bad, Windows home
```

Pro-tips

To access Windows files from Linux, navigate to:

```
cd /mnt/c/
```

For example, here is how to access your Windows Desktop from WSL:

```
cd /mnt/c/Users
ls
cd <your-username>
ls
cd Desktop
```

```

For example, here's how to access your Windows Desktop.  Your username will be different.
```console
$ cd /mnt/c/Users
$ ls
'All Users'  'Default User'   defaultuser0   awdeorio   Default   Public   defaultuser100000   desktop.ini
$ cd awdeorio
$ ls
... Desktop  Downloads ...
$ cd Desktop
```

Copy/Paste functionality in your bash terminal may be enabled by default with the usual `Ctrl + C` and `Ctrl + V` keyboard shortcuts.

Or, try `Ctrl + Shift + C` and `Ctrl + Shift + V`.

If these do not work, right click on the terminal window header. If you see **"Settings"**, go to **Settings -> Actions** and add your desired keybindings for copy/paste.

If you see **"Properties"**, go to **Properties -> Options** and ensure **"Use Ctrl+Shift+C/V as Copy/Paste"** is checked.

Microsoft Troubleshooting

Microsoft maintains an extensive troubleshooting page for WSL installation. If you're getting a specific error code or message, you may be able to match it with a resolution there.

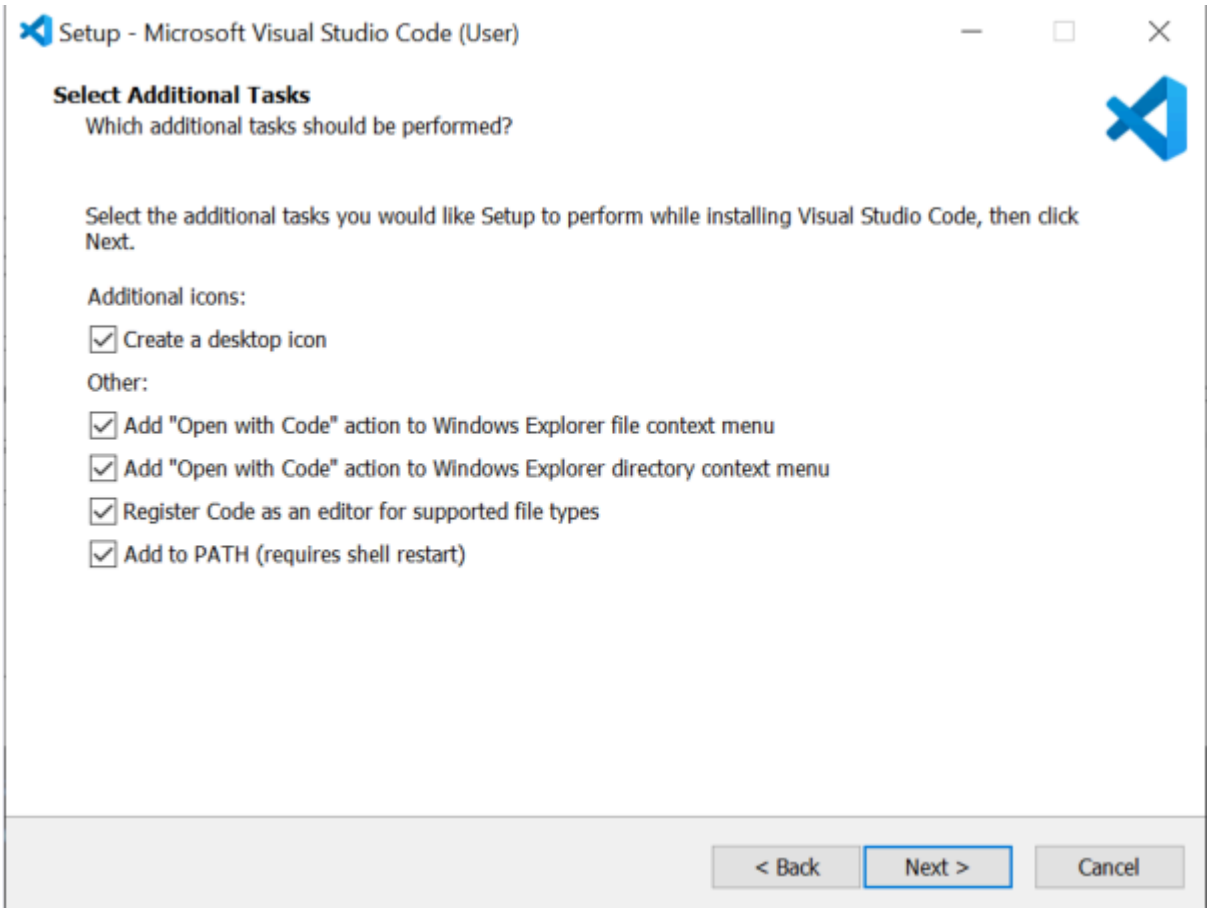
If you forgot your WSL password, follow Microsoft's reset-password steps.

If you are running an older version of Windows 10 and are unable to update to build 19041 or higher, you may be able to perform a manual installation. Use this only as a last resort.

Setup VS Code for C/C++ on Windows

Install

Select each of the options below during installation.
```



WSL Extension

Install the **WSL Extension**, which allows the VS Code backend to run in WSL where the C++ compiler lives.

1. Open the extensions panel from the left sidebar.
2. Search for **WSL**.
3. Click **Install**.

VS Code might attempt to install this one automatically. If so, you may see **Installing** followed by a **Reload Window** option. Click that instead of **Install**.

Next, connect to WSL:

1. Click the button in the bottom left corner.
2. Select **"Connect to WSL using Distro..."** from the menu.
3. Choose **"Ubuntu 24.04"** from the list.

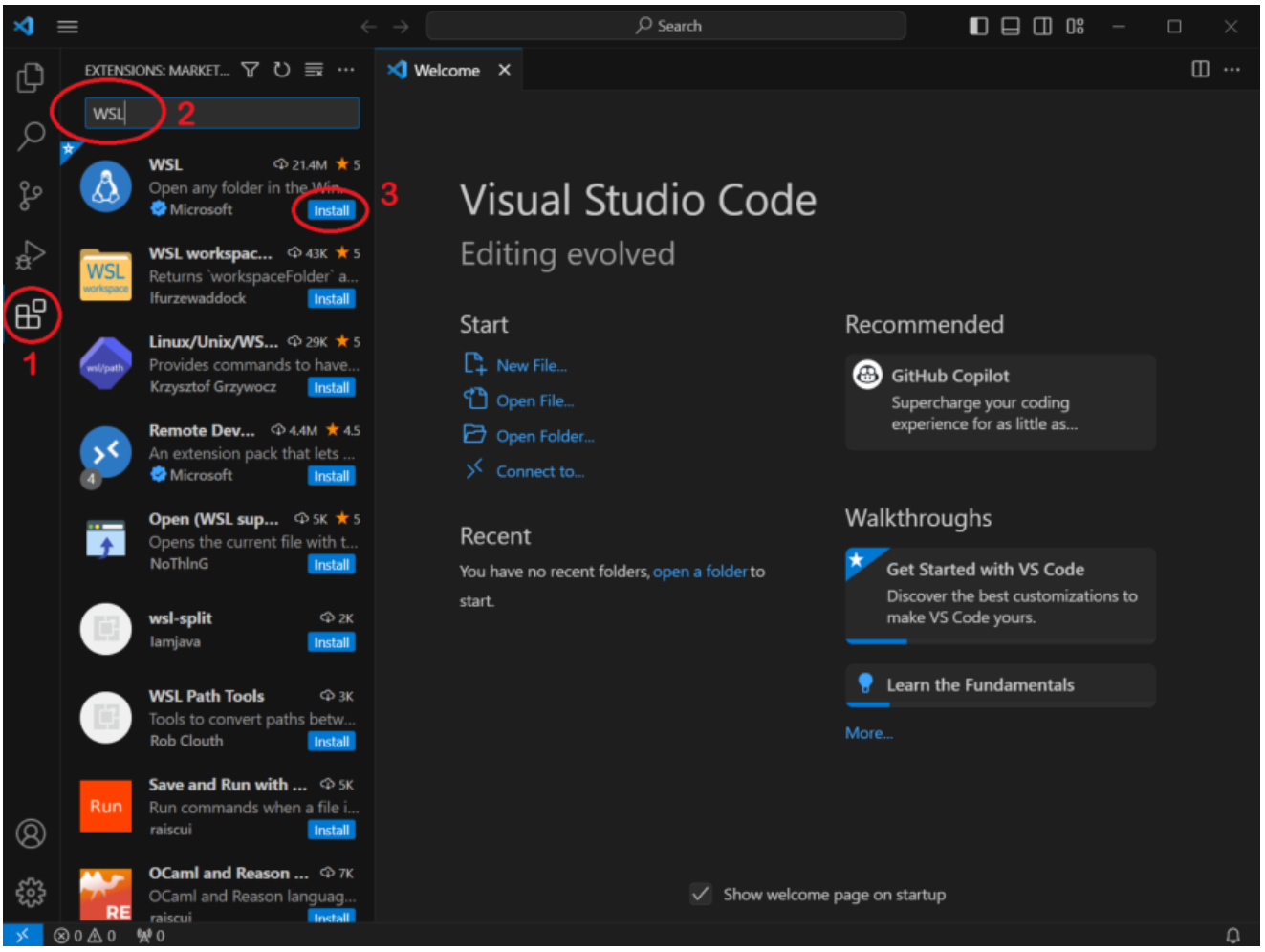
Now, the button in the bottom left should say **"WSL: Ubuntu-24.04"**. For any C++ development, make sure VS Code is always connected to WSL. Depending on your version of VS Code and Windows, your button might be blue instead of green, and you might not have the purple bar at the bottom of your window - that's fine.

Similarly, install the C++ extension to run in WSL.

Note that you need the **"C/C++"** extension. You do **not** need the **"C/C++ Extension Pack"**.

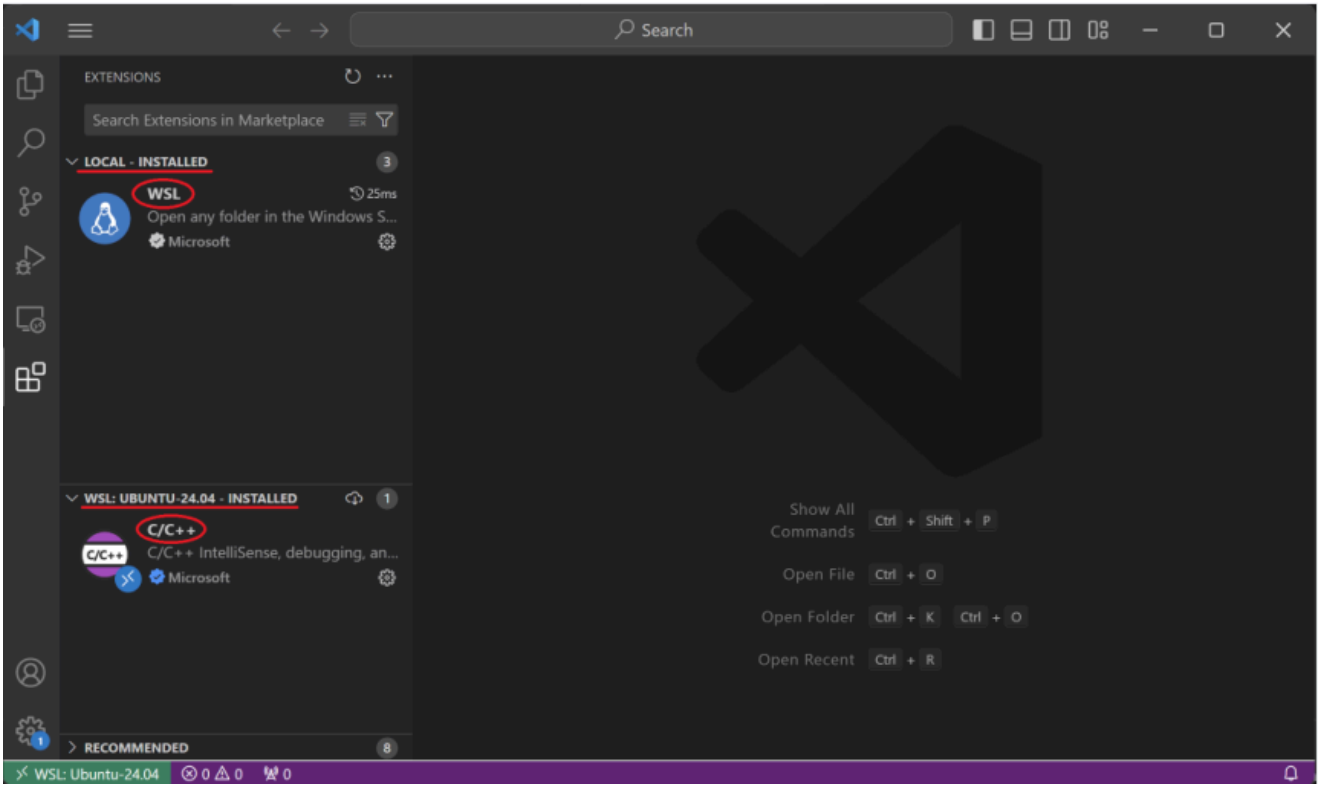
Clear out the search bar in the extensions panel. You should see:

- **WSL installed locally**
- **C/C++ installed in WSL: Ubuntu-24.04**



Integrated Terminal

Finally, open the integrated terminal within VS Code. Go to **Terminal -> New Terminal** (or press `Ctrl + ``). The integrated terminal will automatically use your project directory as its working directory, and using it is easier than switching back and forth to your separate terminal window.



Reference

This tutorial is made by **Yuzhe Niu** in **2025SU**.
Edited by **Siqi Cao** in **2026SP**.
It takes reference from the following:

1. Tech-JI/Linux-Install-Party
2. UM EECS280 tutorials
3. ECE2800J 2025SP